

Compiler Principle

Text Book: Modern Compiler Implementation in C

Author: Andrew W. Appel

2024 Spring&Summer

Content

1. INTRODUCTION
2. **LEXICAL ANALYSIS**
3. PARSING
4. ABSTRACT SYNTAX
5. SEMANTIC ANALYSIS
6. ACTIVATION RECORD
7. TRANSLATING INTO INTERMEDIATE CODE
8. BASIC BLOCKS AND TRACES
9. INSTRUCTION SELECTION
10. LIVENESS ANALYSIS
11. REGISTER ALLOCATION
13. GARBAGE COLLECTION
14. OBJECT-ORIENTED LANGUAGES
18. LOOP OPTIMIZATIONS

Overview

Compiling Process:

To translate a program from one language into another, a compiler **first pull it apart** and **understand its structure and meaning**, then **put it together** in a different way.

The **front end** of the compiler performs analysis;

The **back end** does synthesis.

IR generation separate the front end and the back end.

Overview

The analysis is usually broken up into

Lexical analysis:

breaking the input into individual words or "tokens";

Syntax analysis:

parsing the phrase structure of the program;

Semantic analysis:

calculating the program's meaning.

Overview

- **Task** of lexical analyzer
 - **Taking** a stream of characters
 - **Producing** a stream of tokens
 - *names, keywords, and punctuation marks*
 - **Discarding** *white space and comments*

```
float match0(char *s) /* find a zero */
{ if (!strcmp(s, "0.0", 3))
    return 0.;
}
```

FLOAT	ID(match0)	LPAREN	CHAR	STAR	ID(s)
RPAREN	LBRACE	IF	LPAREN	BANG	
ID(strcmp)	LPAREN	ID(s)	COMMA	STRING(0.0)	
COMMA	NUM(3)	RPAREN	RPAREN	RETURN	
REAL(0.0)	SEMI	RBRACE	EOF		

Overview

The interface of lexical analysis is a function (e.g., *getToken*) that returns the next token after being called.

How do we discuss lexical analysis?

- Attack it with high-powered **formalisms** and **tools**
- Why?
 - Similar **formalisms** will be **useful in the study of parsing**
 - Similar **tools** have many applications in **areas other than compilation**

2.1 Lexical Token

Lexical Analyzer

A stream of
characters



Lexical Analyzer



A stream of
lexical tokens

**What does a lexical
token mean?**

Lexical Tokens

- A lexical token:
 - **A sequence of characters**
 - **A unit in the grammar** of a programming language
- Classification of lexical tokens: **A finite set** of token types

Type	Examples
ID	foo n14 last
NUM	73 0 00 515 082
REAL	66.1 .5 10. 1e67 5.5e-10
IF	if
COMMA	,
NOTEQ	!=
LPAREN	(
RPAREN	)

Lexical Tokens

- **Reserved words**, in most languages, are **not used as identifiers**
 - e.g., IF, VOID, RETURN
- Examples of **non-tokens**:

comment

`/* try again */`

preprocessor directive

`#include<stdio.h>`

preprocessor directive

`#define NUMS 5, 6`

macro

`NUMS`

blanks, tabs, and new-lines

- The preprocessor deletes the non-tokens
 - Operates on the source character stream
 - Producing another character stream to the lexical analyzer

Lexical Tokens

- Given a **program** such as

```
float match0(char *s) /* find a zero */
{ if (!strcmp(s, "0.0", 3))
    return 0.;
}
```

- The **lexical analyzer** will return the stream:

FLOAT	ID(match0)	LPAREN	CHAR	STAR	ID(s)
RPAREN	LBRACE	IF	LPAREN	BANG	
ID(strcmp)	LPAREN	ID(s)	COMMA	STRING(0.0)	
COMMA	NUM(3)	RPAREN	RPAREN	RETURN	
REAL(0.0)	SEMI	RBRACE	EOF		

- The **token-type** of each token is reported
- Some of the tokens **attached semantic values**
 - Such as identifiers and literals, with **auxiliary information**

Lexical Tokens

- Now we have known the input and output. **How to implement a lexical analyzer?**
- To implement a lexical analyzer, we first need to **describe the lexical rules** of the source programming languages.
 - **How?**

Lexical Tokens

Describing the lexical tokens of a programming language in English.

Example: identifiers in C or Java:

1. An identifier is a sequence of letters and digits; the first character must be a letter
2. The underscore _ counts as a letter
3. Upper- and lowercase letters are different
4. If the input stream has been parsed into tokens up to a given character, the next token is taken to include the longest string of characters that could possibly constitute a token
5. Blanks, tabs, newlines, and comments are ignored except as they serve to separate tokens
6. Some white space is required to separate otherwise adjacent identifiers, keywords, and constants

Lexical Tokens

How to implement a lexical analyzer based on lexical rules?

- An **ad hoc** lexer
 - Any reasonable programming language can be used to implement it
- A **simpler and more readable** lexical analyzers
 - **Regular expressions**: specify lexical tokens
 - easy to understand but hard to implement
 - **Deterministic finite automata**: implement lexers.
 - Easy to implement, but not easy to manually construct from specifications
 - **Mathematics**: connect the above two

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

2.2 Regular Expression

Some Concepts

- A **language** is a set of strings
- A **string** is a finite sequence of symbols
- A **symbol** is taken from a finite alphabet

C language

infinite

The language of C reserved words

Notices:

- **Not assign any meaning** to the strings;
- **Only** classify each string as **in the language or not**.

Regular Expression

Regular expressions:

- Can specify some of these (possibly infinite) languages with **finite descriptions**.
- Each regular expression stands for a set of strings

Each regular expression **r** denotes a regular language **$L(r)$**

Regular Expression - Definition

A regular expression (RE) is defined inductively:

Symbol: a

- For each symbol a in the alphabet of the language
- The **regular expression** a : denotes the language containing just **the string** a

Epsilon: ϵ

- The regular expression ϵ represents a **language** whose only **string is the empty string**.
- i.e.,: $L(\epsilon) = \{""\}$.

Regular Expression - Definition

Suppose M and N are two regular expressions:

Alternation: A vertical bar |

- M/N is a regular expression
- $L(M|N) = \{ s \mid s \in L(M) \text{ or } s \in L(N) \}$
- Example: $L(a/b)$ contains the two strings a and b

Concatenation: operator $\cdot$

- $M \cdot N$ is a regular expression
- $L(M \cdot N) = \{ st \mid s \in L(M) \text{ and } t \in L(N) \}$
- Example: The regular expression $(a/b) \cdot a$ defines the language containing the two strings aa and ba

Regular Expression - Definition

Repetitions:

- The Kleene closure of M is M^* .
- A **string** is in M^* if it is the **concatenation of zero or more** strings, all of which are in M .
- Example: $((a \mid b) \cdot a)^*$ represents the infinite set $\{ "", "aa", "ba", "aaaa", "baaa", "aaba", "baba", "aaaaaa", \dots \}$.

Regular Expression - Examples

- Using **symbols**, **epsilon**, **alternation**, **concatenation**, and **Kleene closure**, we can specify the set of ASCII characters corresponding to the lexical tokens of a programming language

More examples:

$(0 \mid 1)^* \cdot 0$?

Regular Expression - Examples

- Using **symbols**, **epsilon**, **alternation**, **concatenation**, and **Kleene closure**, we can specify the set of ASCII characters corresponding to the lexical tokens of a programming language

More examples:

$(0 \mid 1)^* \cdot 0$

Binary numbers that are multiples of two.

$b^*(abb^*)^*(a \mid \epsilon)$?

Regular Expression - Examples

- Using **symbols**, **epsilon**, **alternation**, **concatenation**, and **Kleene closure**, we can specify the set of ASCII characters corresponding to the lexical tokens of a programming language

More examples:

$(0 \mid 1)^* \cdot 0$

Binary numbers that are multiples of two.

$b^*(abb^*)^*(a \mid \epsilon)$

Strings of a's and b's **with no consecutive a's**.

$(a \mid b)^* aa (a \mid b)^*$

?

Regular Expression - Examples

- Using **symbols**, **epsilon**, **alternation**, **concatenation**, and **Kleene closure**, we can specify the set of ASCII characters corresponding to the lexical tokens of a programming language

More examples:

$(0 \mid 1)^* \cdot 0$

Binary numbers that are multiples of two.

$b^*(abb^*)^*(a \mid \epsilon)$

Strings of a's and b's **with no consecutive a's**.

$(a \mid b)^* aa (a \mid b)^*$

Strings of a's and b's **containing consecutive a's**.

Regular Expression - Abbreviations

In writing regular expressions, sometimes the **concatenation symbol ($\cdot$)** or **the epsilon (ϵ)** will be omitted

Assuming that

- Kleene closure "**binds tighter**" than concatenation
- Concatenation **binds tighter** than alternation;

So that

- ab / c means $(a \cdot b) / c$, and $(a /)$ means (a / ϵ)

Regular Expression - Abbreviations

Introducing some more abbreviations:

- $[abcd]$ means $(a \mid b \mid c \mid d)$,
- $[b-g]$ means $[bcdefg]$,
- $[b-gM-Qkr]$ means $[bcdefgMNOPQkr]$,
- $M?$ means $(M \mid \epsilon)$, and M^+ means $(M \cdot M^*)$.

None extend the descriptive power of regular expressions

- Any set of strings that can be described with these abbreviations could also be described by just the basic set of operators.

Regular Expression - Summary

a	An ordinary character stands for itself.
ε	The empty string. Another way to write the empty string.
$M \mid N$	Alternation, choosing from M or N .
$M \cdot N$	Concatenation, an M followed by an N .
MN	Another way to write concatenation.
M^*	Repetition (zero or more times).
M^+	Repetition, one or more times.
$M?$	Optional, zero or one occurrence of M .
$[a - zA - Z]$	Character set alternation.
$.$	<i>A period stands for any single character except newline.</i>
$"a.+*"$	<i>Quotation, a string in quotes stands for itself literally.</i>

Figure 2.1: Regular expression notation.

Regular Expressions for Tokens

if	{return IF;}
[a-z][a-z0-9]*	{return ID;}
[0-9]+	{return NUM;}
([0-9]+"."[0-9]*) ([0-9]*"."[0-9]+)	{return REAL;}
("--"[a-z]*"\n") (" " "\n" "\t")+	{/*do nothing*/}
.	{ error();}

Figure 2.2: Regular expressions for some tokens

- The **fifth line** of the description recognizes comments or white space but does not report back to the parser.
- **Instead:** The white space is discarded and the lexer resumed
- The **comments** begin with two dashes, contain only alphabetic characters, and end with new-line.

Regular Expressions for Tokens

if	{return IF;}
[a-z][a-z0-9]*	{return ID;}
[0-9]+	{return NUM;}
([0-9]+"."[0-9]*) ([0-9]*"."[0-9]+)	{return REAL;}
("--"[a-z]*"\n") (" " "\n" "\\t")+	{/*do nothing*/}
.	{ error();}

Figure 2.2: Regular expressions for some tokens

- A lexical specification should be complete
 - A rule that matches any single character (and in this case, prints an “illegal character” error message and continues).

Regular Expressions for Tokens

if	{return IF;}
[a-z][a-z0-9]*	{return ID;}
[0-9]+	{return NUM;}
([0-9]+"."[0-9]*) ([0-9]*"."[0-9]+)	{return REAL;}
("--"[a-z]*"\n") (" " "\n" "\t")+	{/*do nothing*/}
.	{ error();}

- These rules are **a bit ambiguous**.
- For example:
 - Does **if8** match as a **single identifier** or as the **two tokens if** and **8**?
 - Does the string **if 89** begin with an identifier or a reserved word?
- How to solve this problem?

Regular Expressions

Two important disambiguation rules used by Lex, JavaCC, SableCC, and other similar lexical-analyzer generators:

- **Longest match:**

- The longest initial substring of the input that can match any regular expression is taken as the next token.

- **Rule priority:**

- For a particular longest initial substring, **the first regular expression** that can match determines its token-type.
- This means that **the order** of writing down the regular-expression rules **has significance**.

Thus

- **if8** matches as an identifier by the longest-match rule
- **if** matches as a reserved word by rule-priority.

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

2.3 Finite Automata

Finite Automata

Regular expressions:

- Convenient for specifying lexical tokens

Needing a formalism:

- Implemented as a computer program
- Using finite automata (Note the singular of automata is automaton)

Finite Automata

A finite automaton:

- A finite set of **states**;
- **Edges** lead from one state to another, and each edge is **labeled with a symbol** from the input symbol set/alphabet Σ ,
- One state is **the start state**,
- and certain of the states are distinguished as **final states**.

Finite Automata

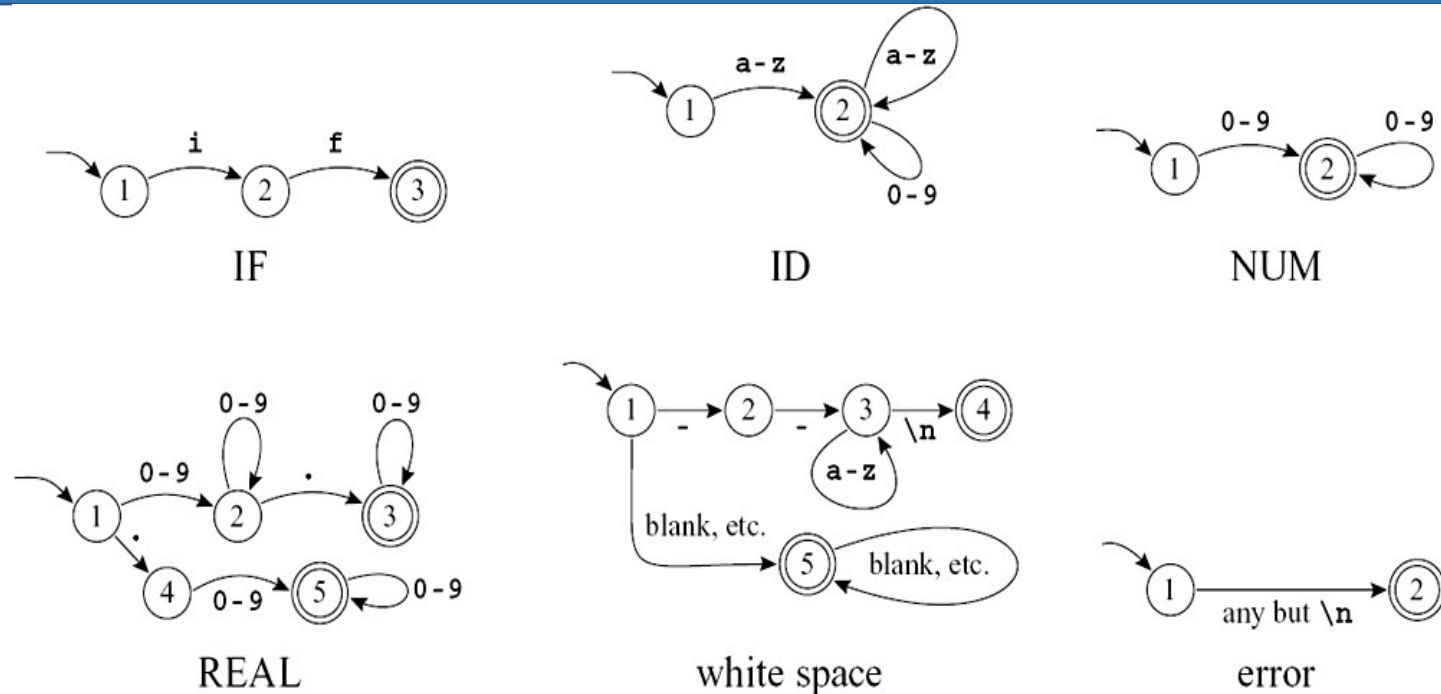
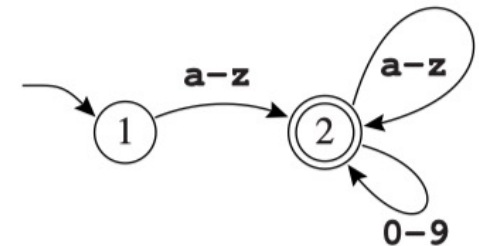


Figure 2.3: Finite automata for lexical tokens.

- The **states** are indicated by circles;
- The **final states** are indicated by double circles.
- The **start state** has an arrow coming in from nowhere.
- An edge **labeled with several characters** is shorthand for many parallel edges.

Finite Automata

In a *deterministic* finite automaton (DFA), **no two edges leaving from the same state** are labeled with the same symbol.



ID

A DFA *accepts* or *rejects* a string as follows:

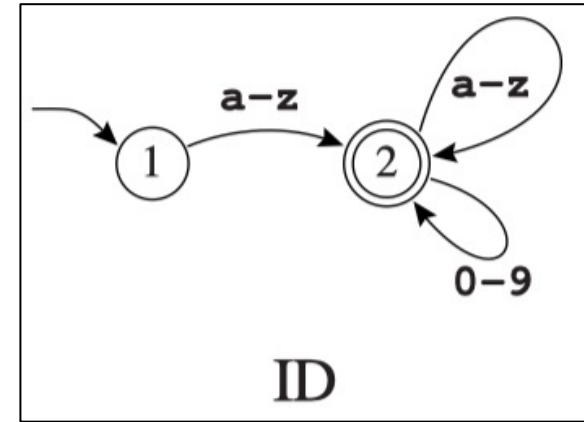
- Starting in the start state, for each character in the input string the automaton follows exactly **one edge** to get to the next state.
- The edge must be **labeled with the input character**.
- After making n transitions for an n -character string, if the automaton is in a final state, then it **accepts** the string.
- If it is not in a final state, or if at some point there was no appropriately labeled edge to follow, it **rejects**.

The language recognized by an automaton is **the set of strings** that it accepts.

Finite Automata

For example

Any string in the language
recognized by automaton ID must
begin with a letter.



Any single letter leads to state 2, which is final; so a single-letter string is accepted.

From state 2, any letter or digit leads back to state 2, so a letter followed by any number of letters and digits is also accepted.

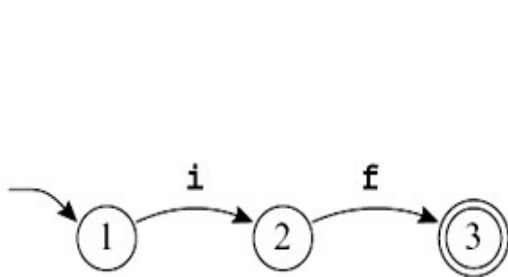
The machine accept the same languages as the following regular expression:

$[a-z][a-z0-9]^*$

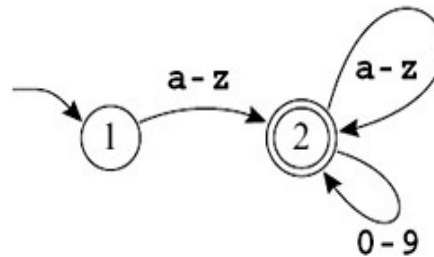
Finite Automata

These are six separate automata; **How to combine them into a single machine that can serve as a lexical analyzer ?**

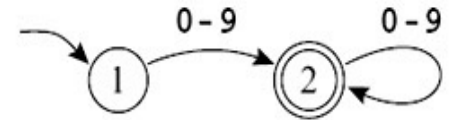
For now, we do it in a ad-hoc way



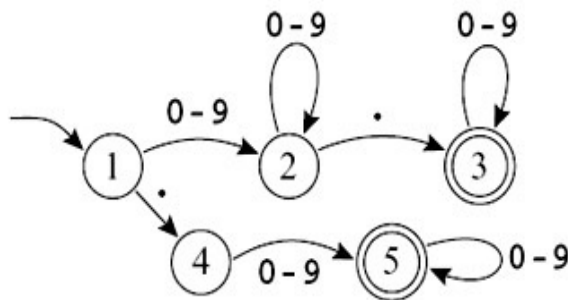
IF



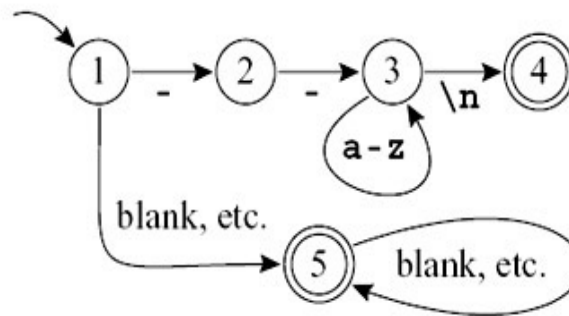
ID



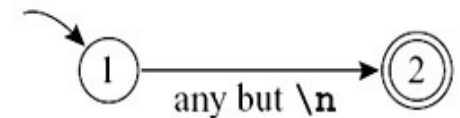
NUM



REAL



white space



error

Finite Automata

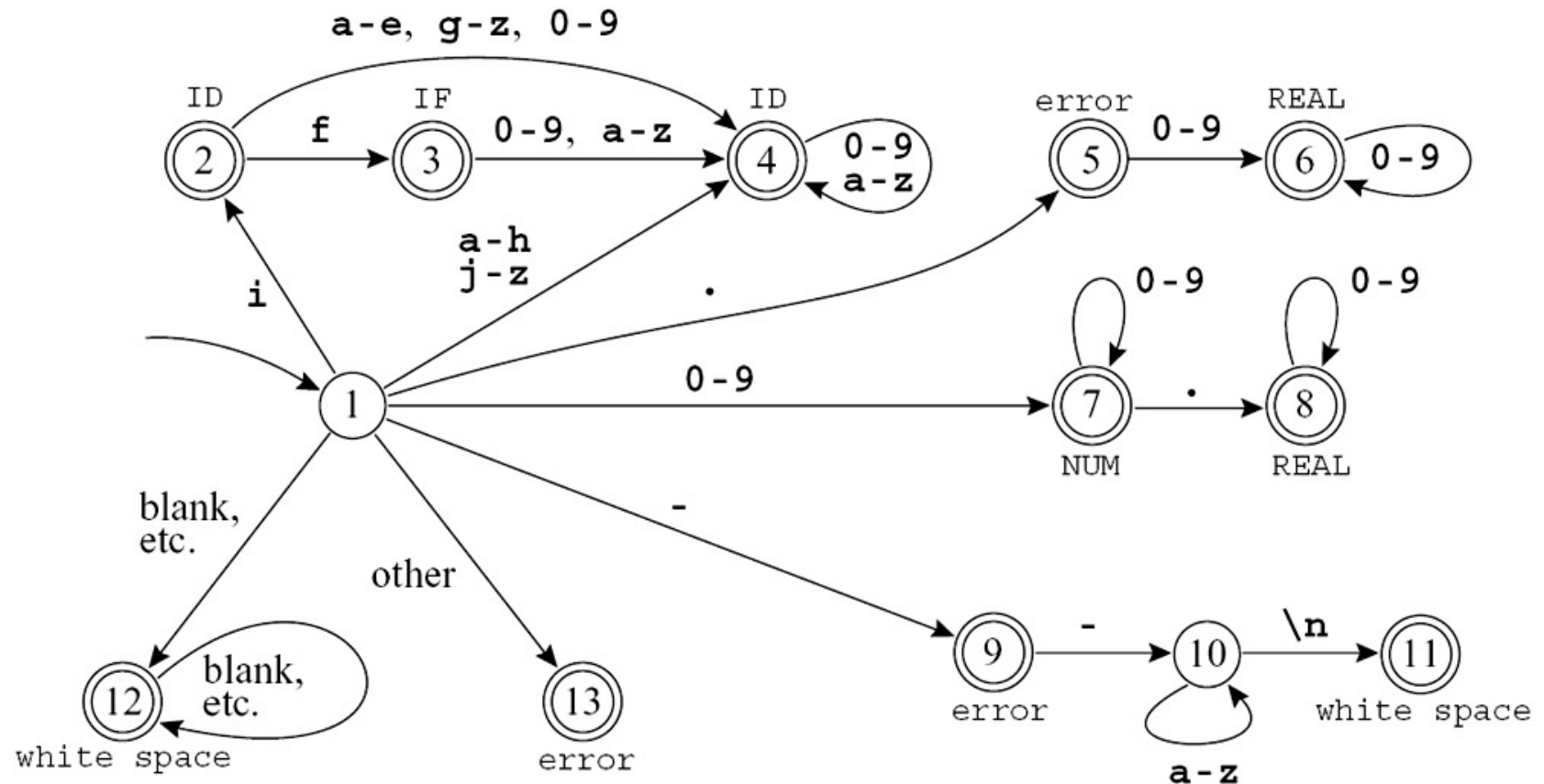


Figure 2.4: Combined finite automaton

- Labeling each final state with the accepted token-type.

Finite Automata

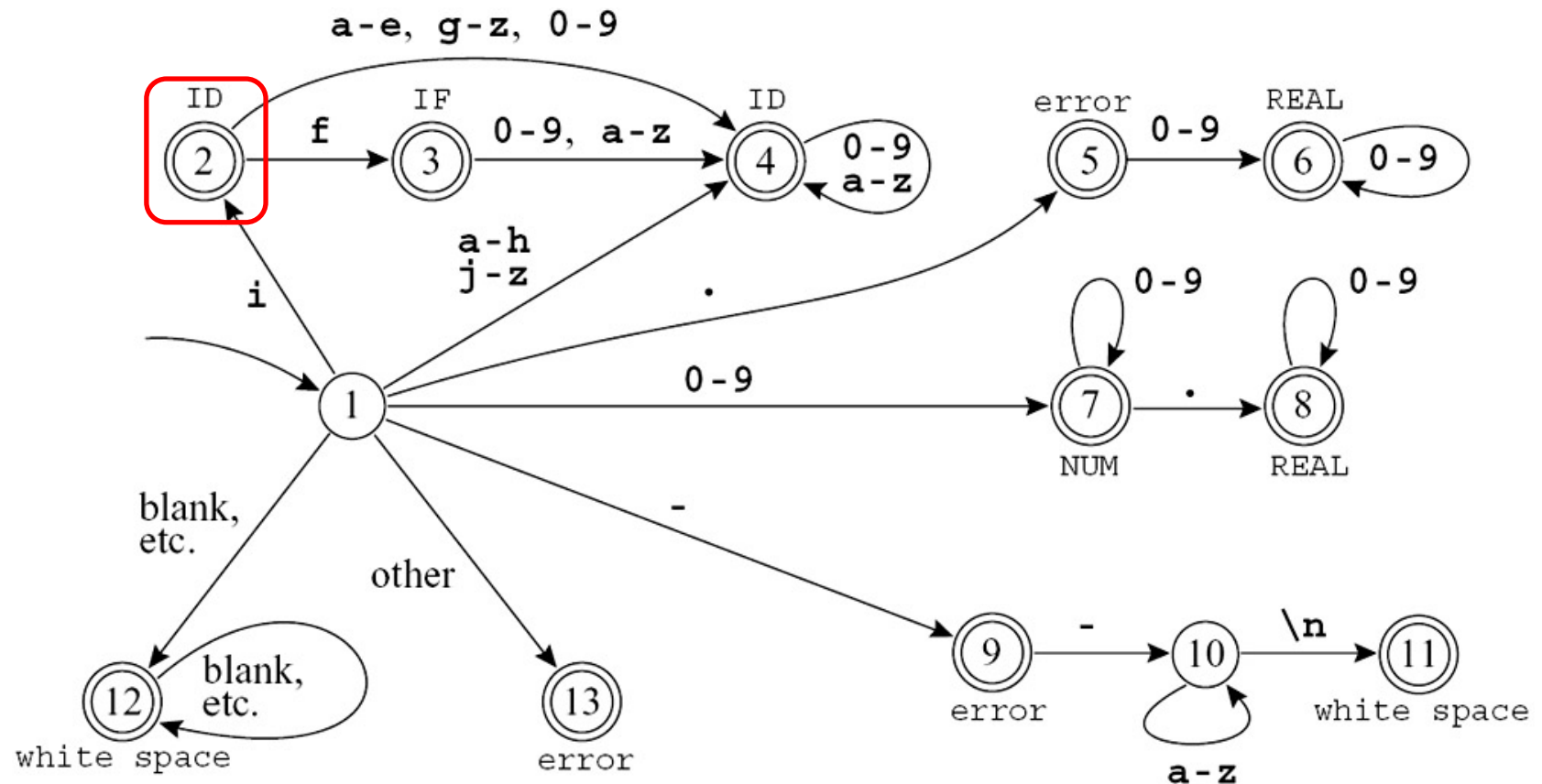


Figure 2.4: Combined finite automaton

- **State 2** has aspects of **state 2** of the IF machine and **state 2** of the ID machine; since the latter is final, then the combined state must be final.

Finite Automata

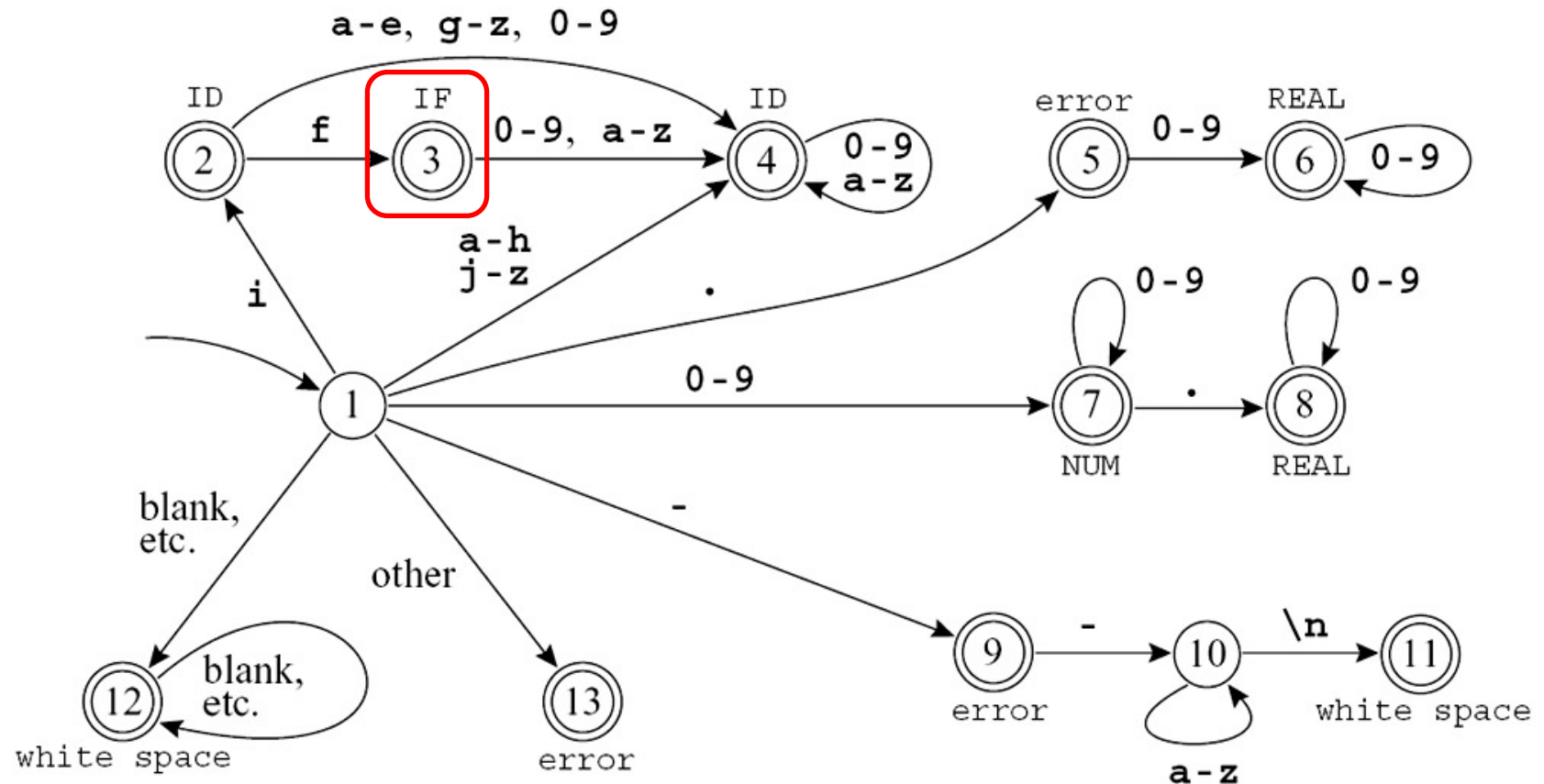


Figure 2.4: Combined finite automaton

- **State 3** is like **state 3** of the IF machine and **state 2** of the ID machine.
- Both are final states. **How to disambiguate?**

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

Finite Automata

Encoding this machine as a **transition matrix**

- subscripted by state number and input character.

There will be a "dead" state (state 0) that loops to itself on all characters; we use this to encode the absence of an edge.

```
int edges[][256]={ /* ...0 1 2...-...e f g h i j... */
/* state 0 */ {0,0,...0,0,0...0...0,0,0,0,0,0...},
/* state 1 */ {0,0,...7,7,7...9...4,4,4,4,2,4...},
/* state 2 */ {0,0,...4,4,4...0...4,3,4,4,4,4...},
/* state 3 */ {0,0,...4,4,4...0...4,4,4,4,4,4...},
/* state 4 */ {0,0,...4,4,4...0...4,4,4,4,4,4...},
/* state 5 */ {0,0,...6,6,6...0...0,0,0,0,0,0...},
/* state 6 */ {0,0,...6,6,6...0...0,0,0,0,0,0...},
/* state 7 */ {0,0,...7,7,7...0...0,0,0,0,0,0...},
/* state 8 */ {0,0,...8,8,8...0...0,0,0,0,0,0...},
    et cetera
}
```

A "finality" array, mapping state numbers to actions

- final state 2 maps to action ID, and so on.

Finite Automata

Recognizing the longest match

- **The job of a lexical analyzer** is to find the longest match:
 - How?
- Keeping track of the longest match with **two variables**
 - **Last-Final** (the state number of the most recent final state encountered)
 - **Input-Position-at-Last-Final**
- Every time a final state is entered, the lexer updates these variables
- When **a dead state** (a nonfinal state with no output transitions) is reached, the variables tell what token was matched and where it ended.

Finite Automata

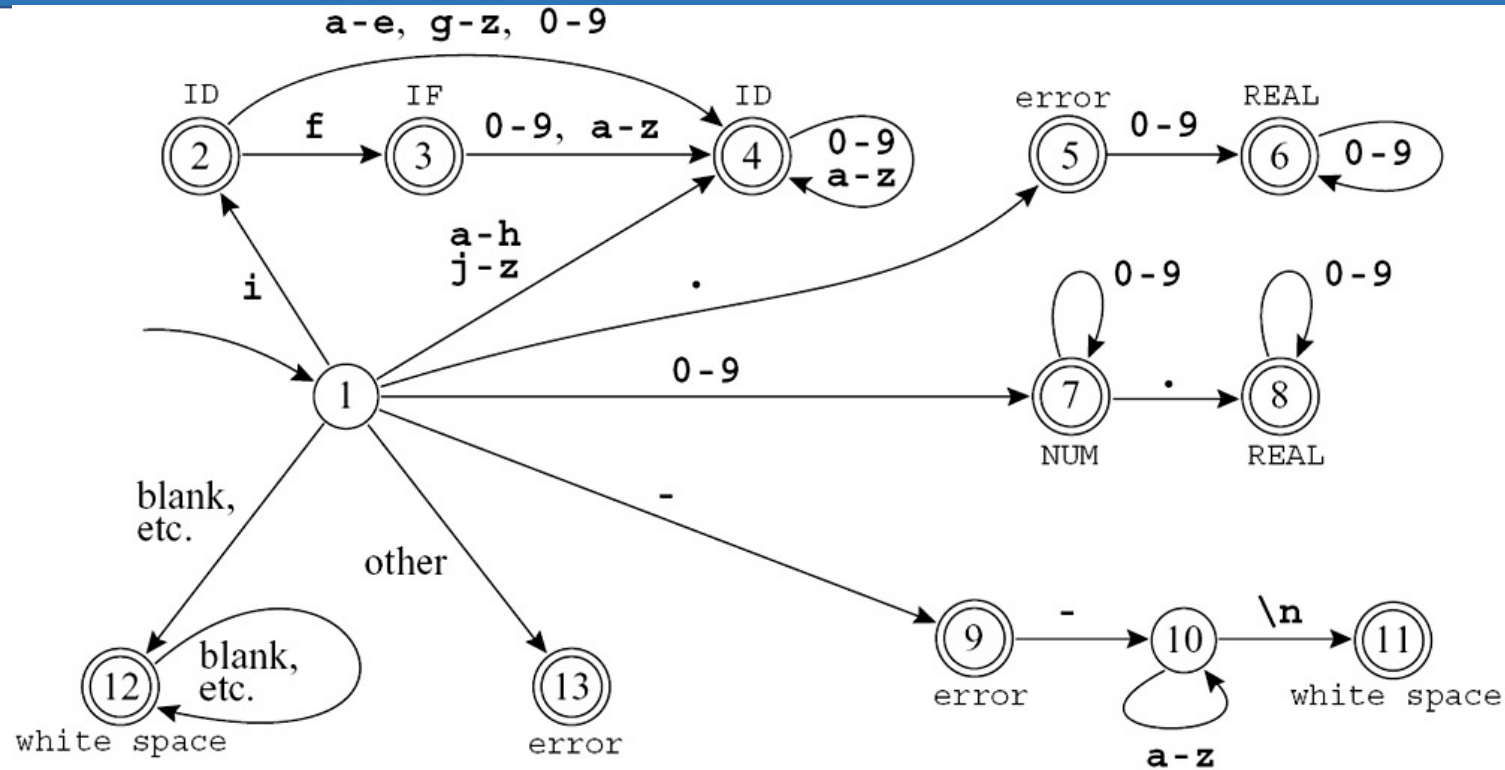


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	<u>I</u> if --not-a-com	

| : input position
 ⊥ : automaton position
 T : Input position at last final

Finite Automata

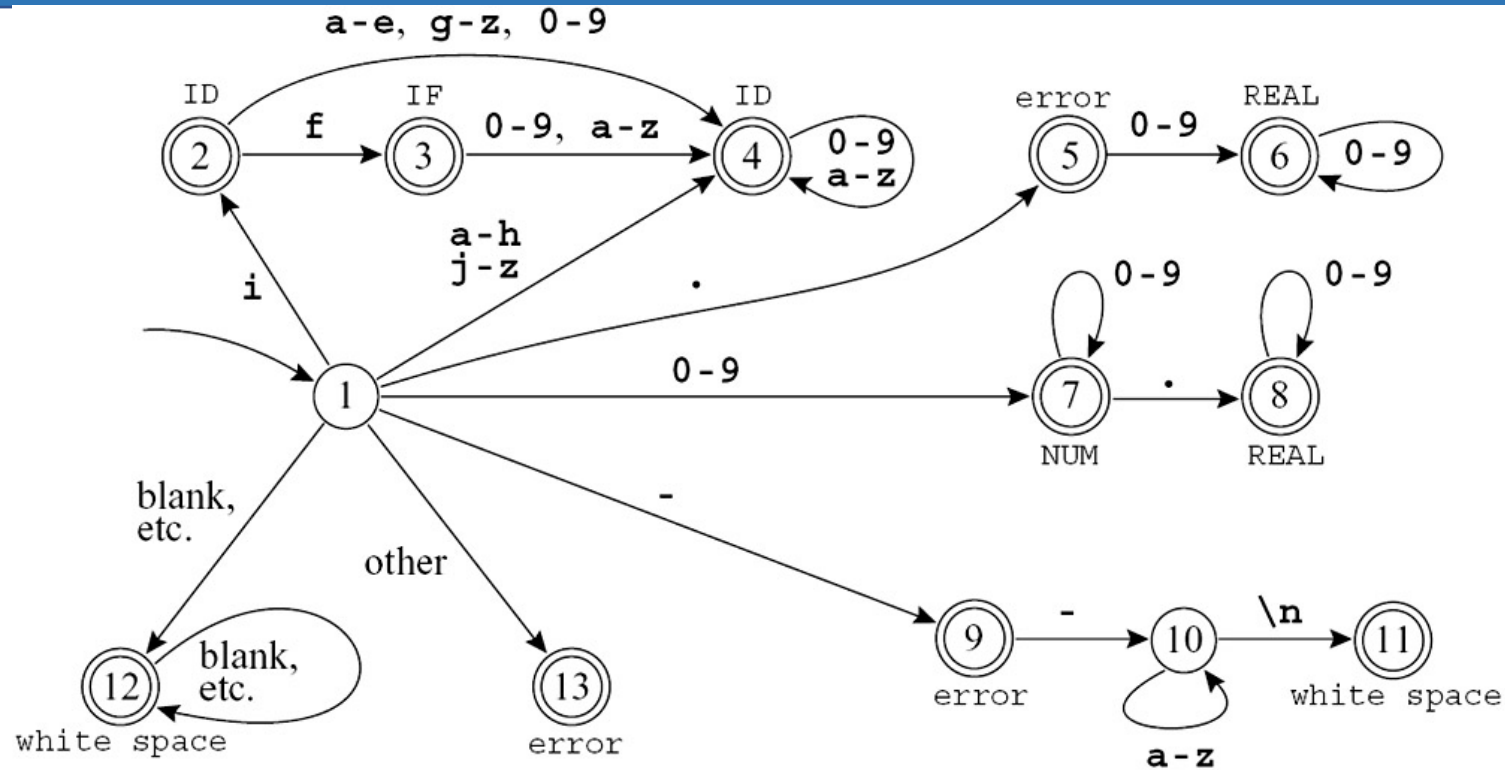


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	<u>I</u> if --not-a-com	
2	2	<u>i</u> if --not-a-com	

| : input position

⊥ : automaton position

T: Input position at last final

Finite Automata

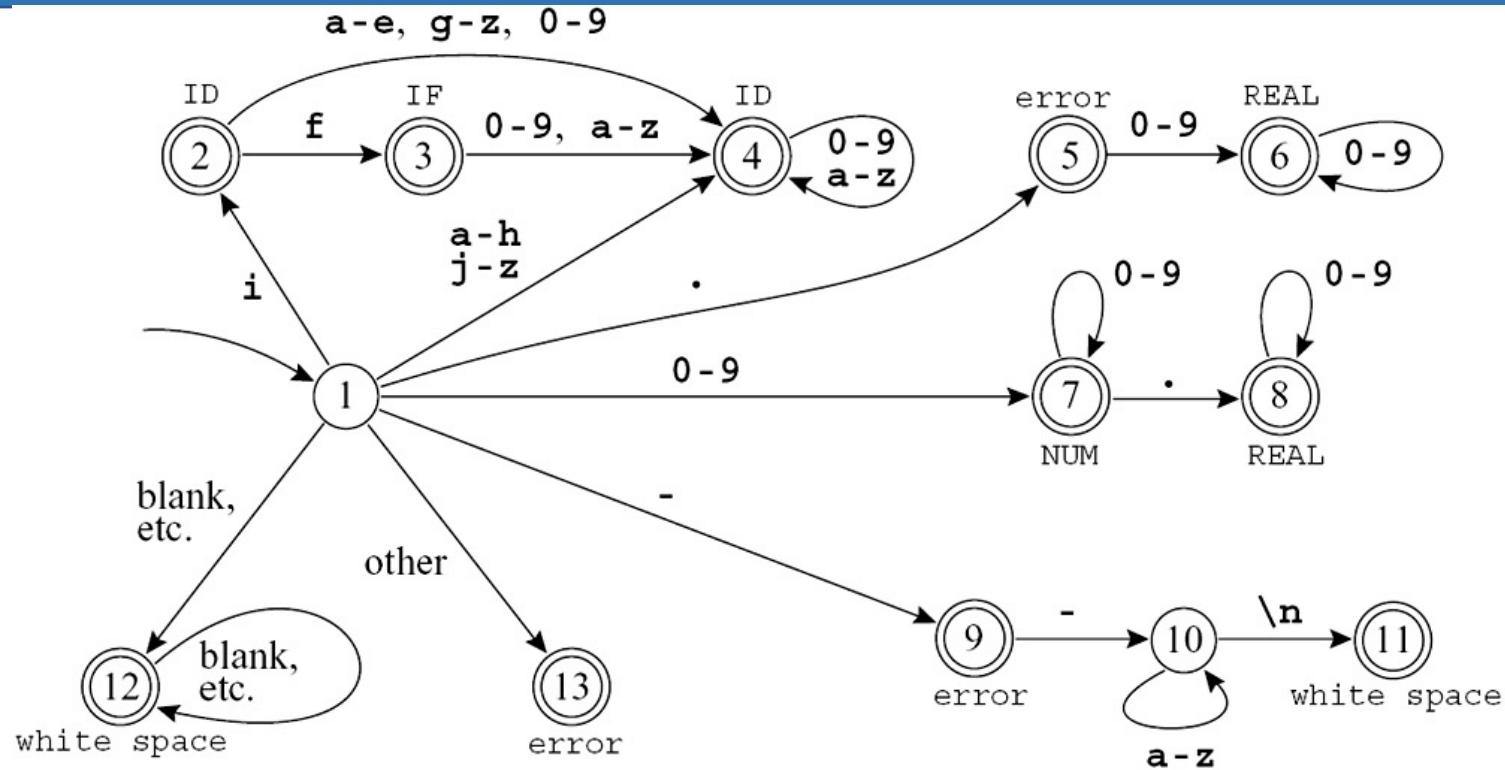


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	<u>I</u> if --not-a-com	
2	2	<u>i</u> f --not-a-com	
3	3	i <u>f</u> --not-a-com	

| : input position

⊥ : automaton position

T: Input position at last final

Finite Automata

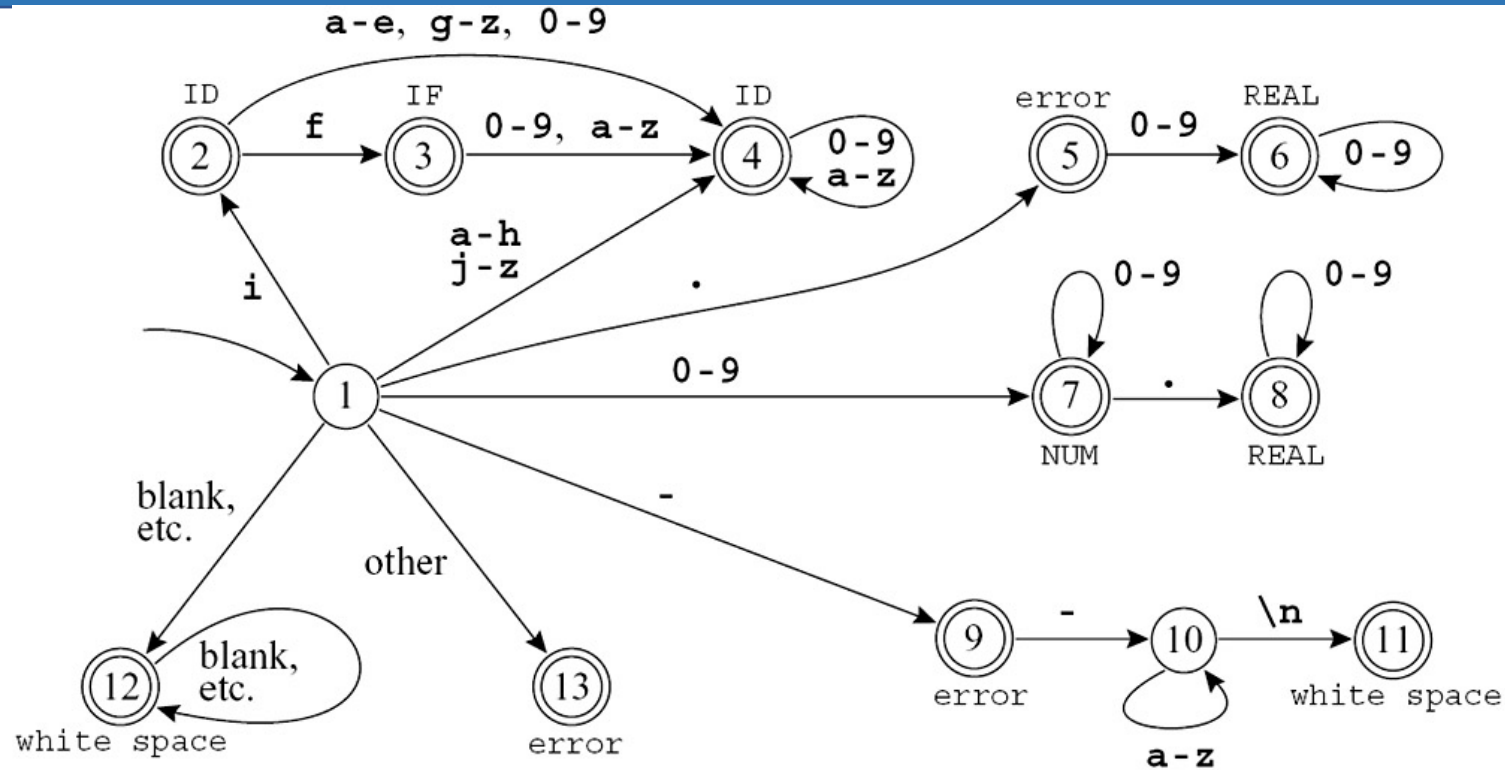


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	<u>i</u> if --not-a-com	
2	2	<u>i</u> f --not-a-com	
3	3	i <u>f</u> --not-a-com	
3	0	if <u>T</u> --not-a-com	return IF

| : input position

⊥ : automaton position

T: Input position at last final

Finite Automata

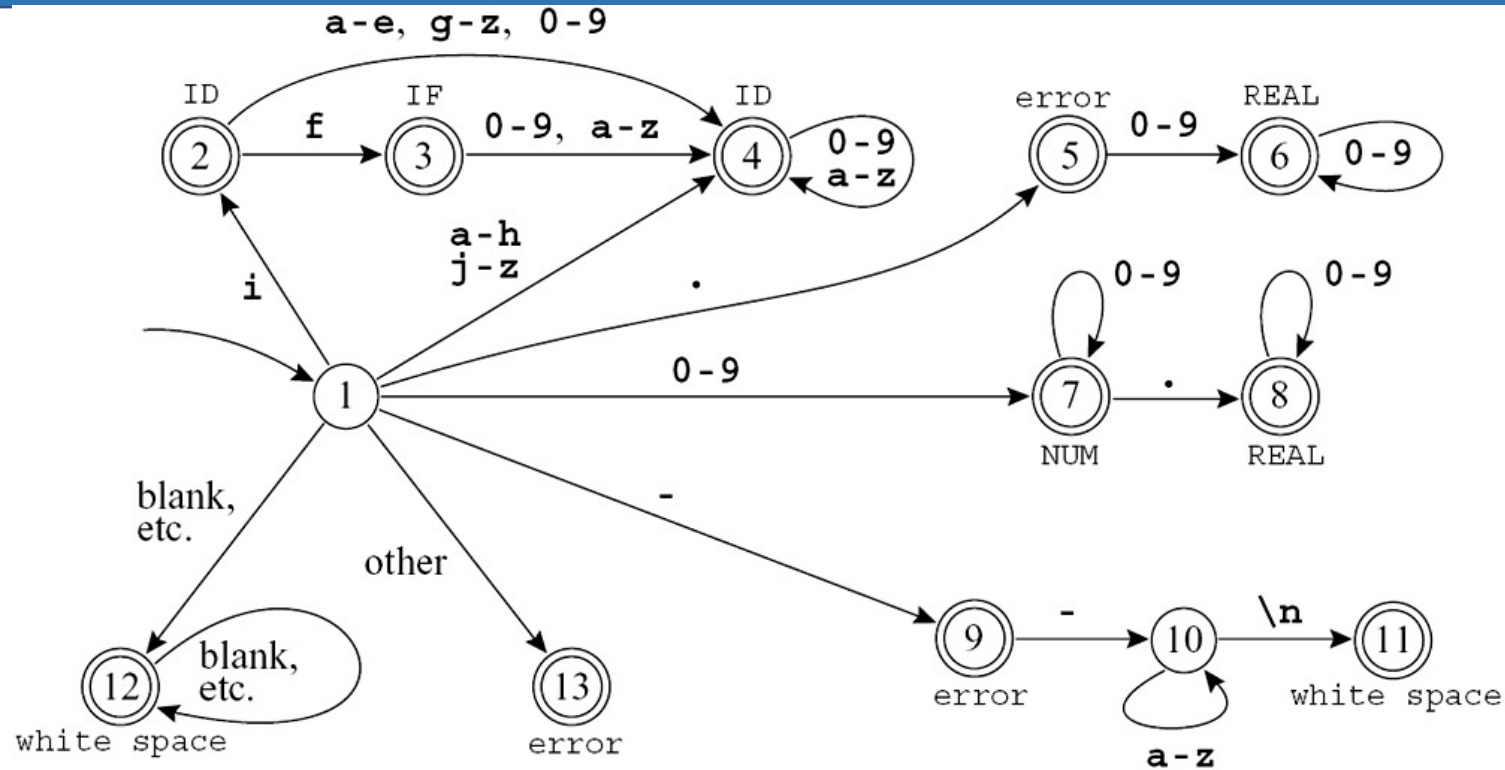


Figure 2.4

|: input position
 ⊥: automaton position
 T: Input position at last final

Last Final	Current State	Current Input	Accept Action
0	1	if --not-a-com	

Finite Automata

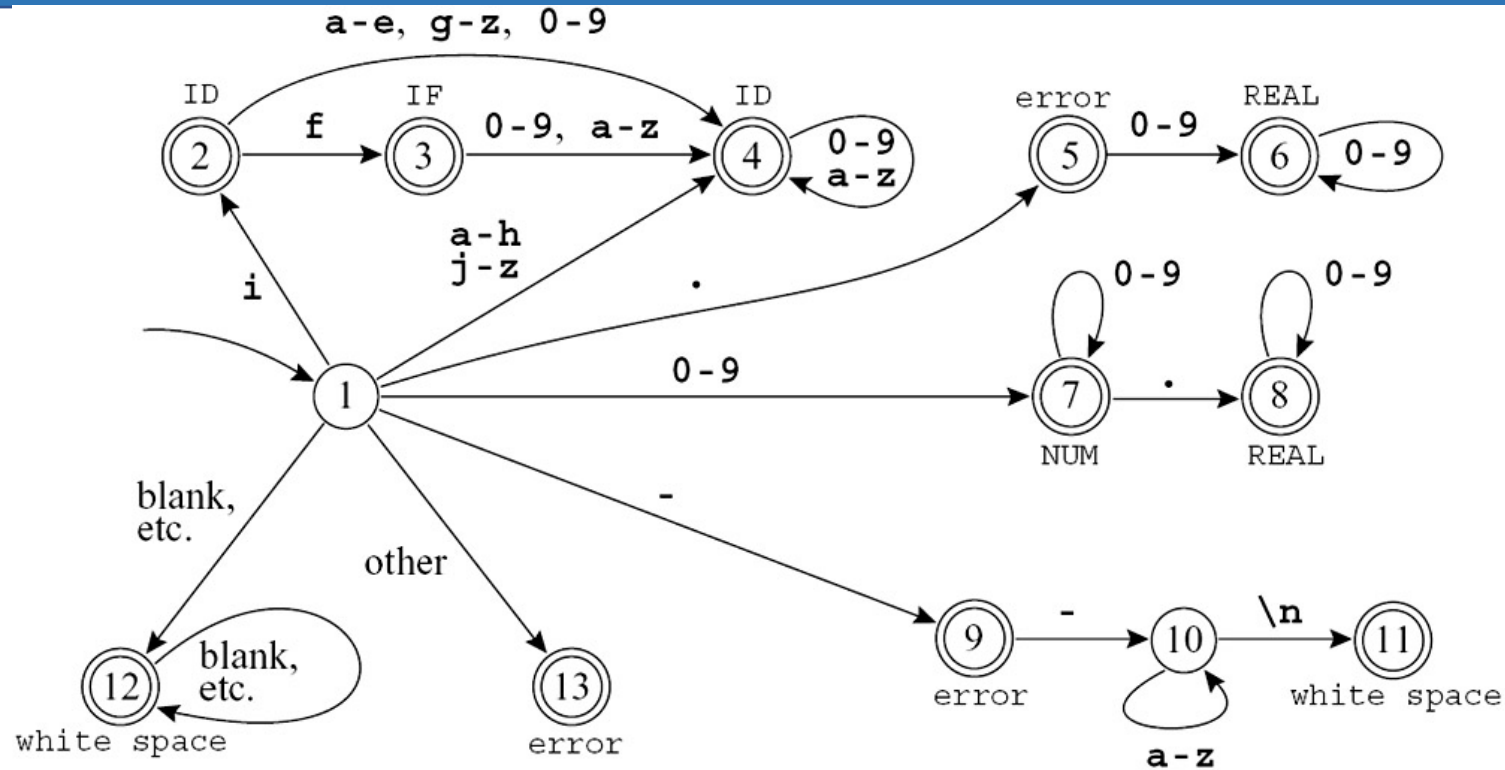


Figure 2.4

| : input position
 ⊥ : automaton position
 T : Input position at last final

Last Final	Current State	Current Input	Accept Action
0	1	if --not-a-com	
12	12	if T --not-a-com	

Finite Automata

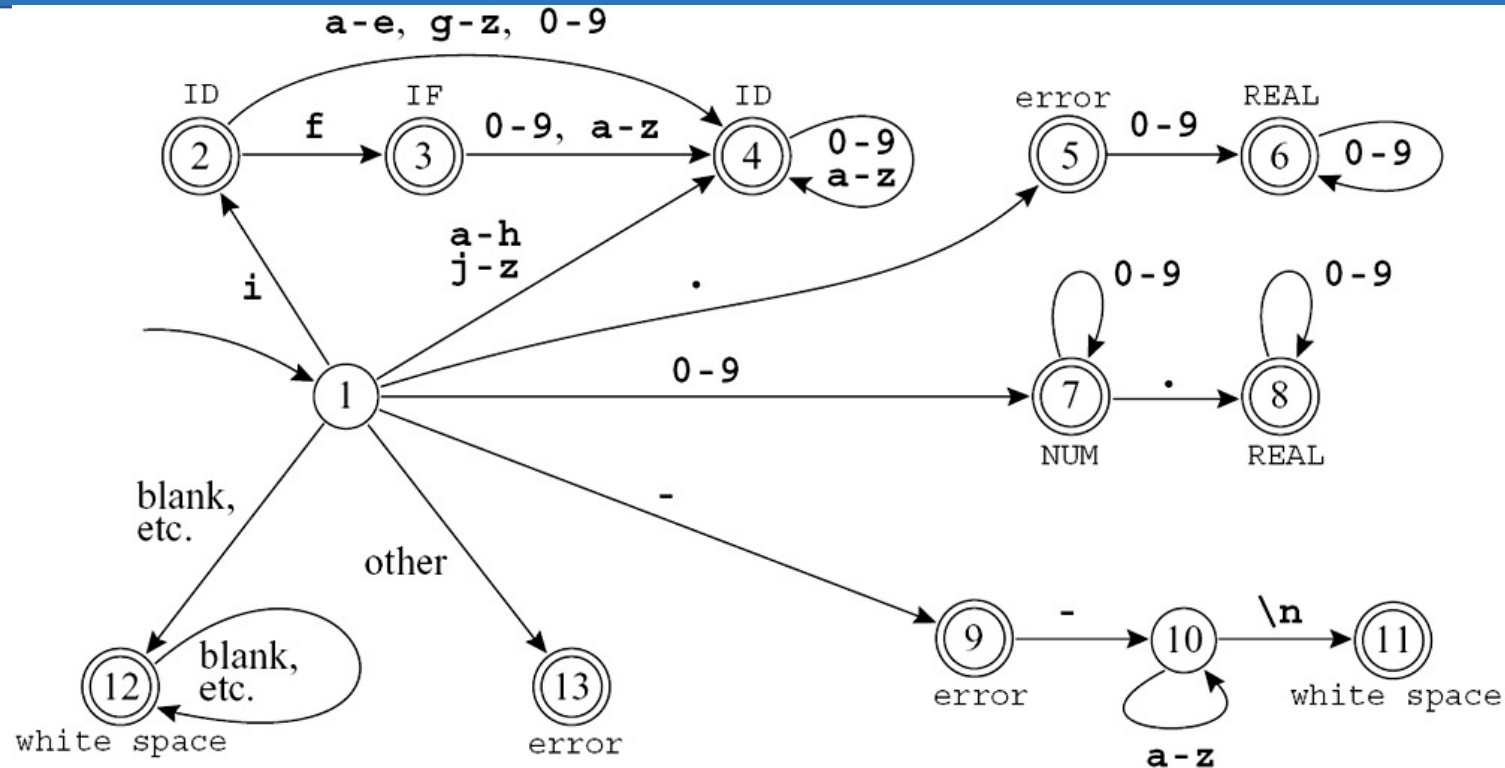


Figure 2.4

| : input position
 ⊥ : automaton position
 T : Input position at last final

Last Final	Current State	Current Input	Accept Action
0	1	if --not-a-com	
12	12	if ⊥ --not-a-com	
12	0	if T ⊥ --not-a-com	<i>found white space; resume</i>

Finite Automata

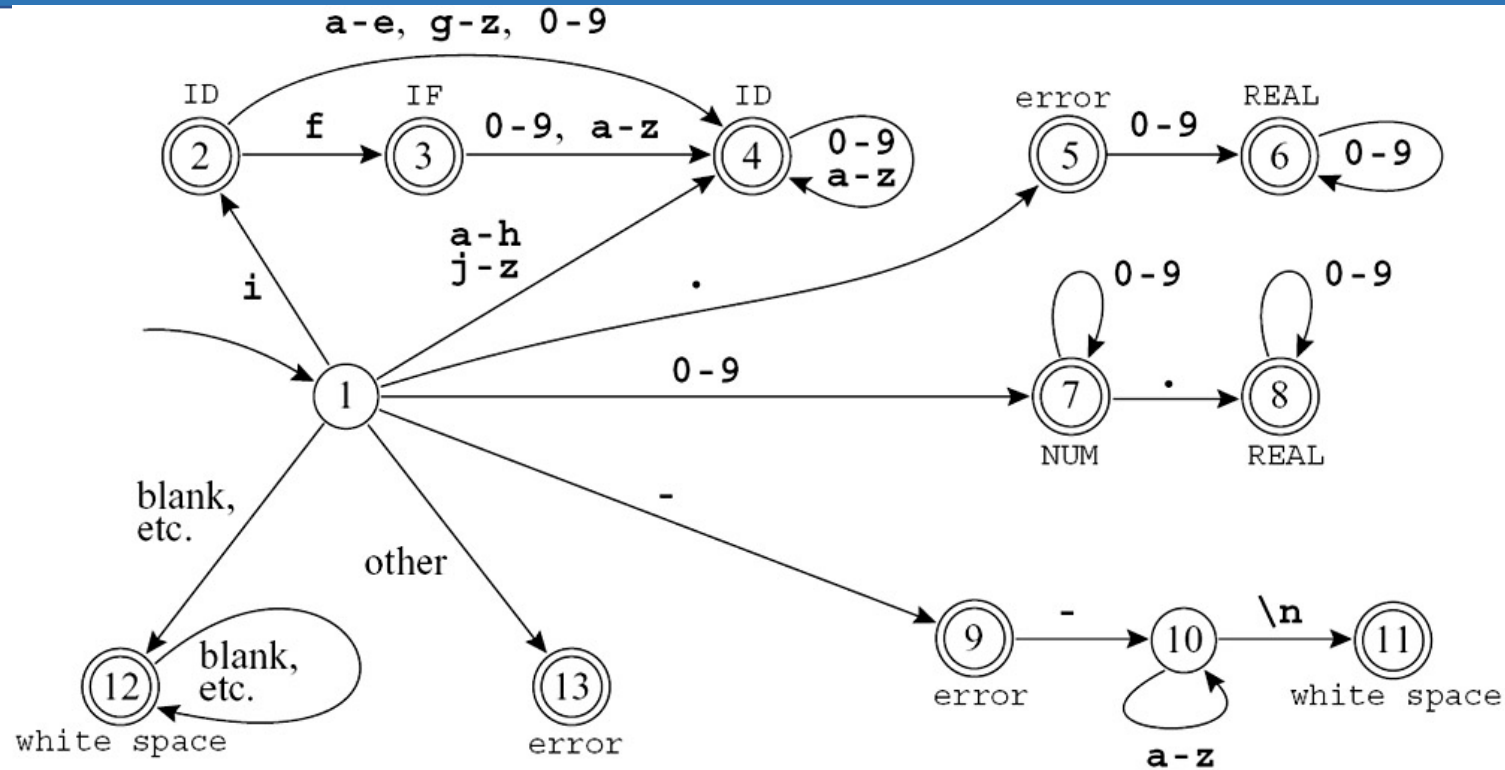


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	if <u>I</u> -not-a-com	
9	9	if <u>I</u> -not-a-com	

| : input position

I : automaton position

T : Input position at last final

Finite Automata

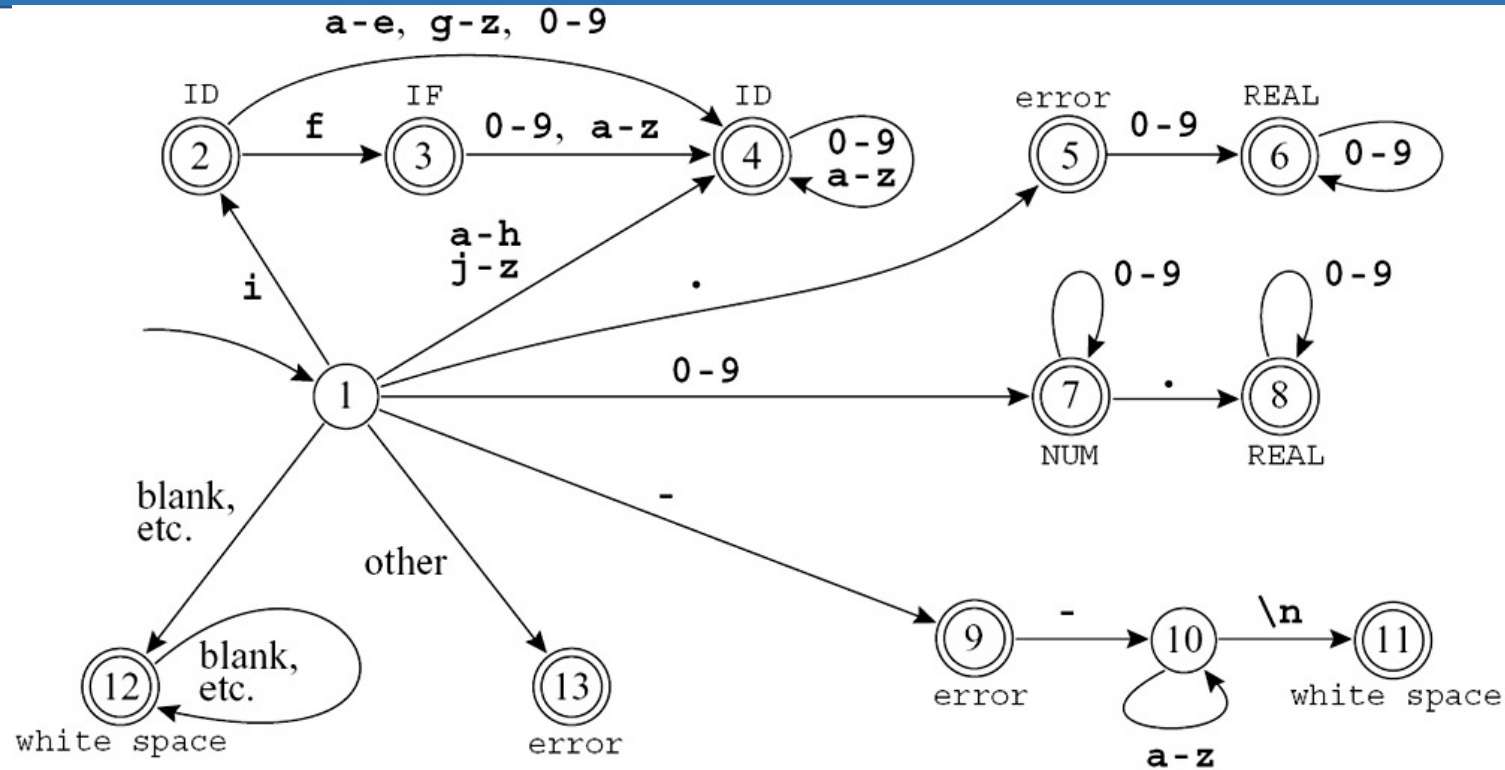


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	if <u>I</u> -not-a-com	
9	9	if <u>I</u> -not-a-com	
9	10	if <u>T</u> -not-a-com	
9	10	if <u>T</u> -not-a-com	
9	10	if <u>T</u> -not-a-com	
9	10	if <u>T</u> -not-a-com	
9	10	if <u>T</u> -not-a-com	

| : input position

I : automaton position

T : Input position at last final

Finite Automata

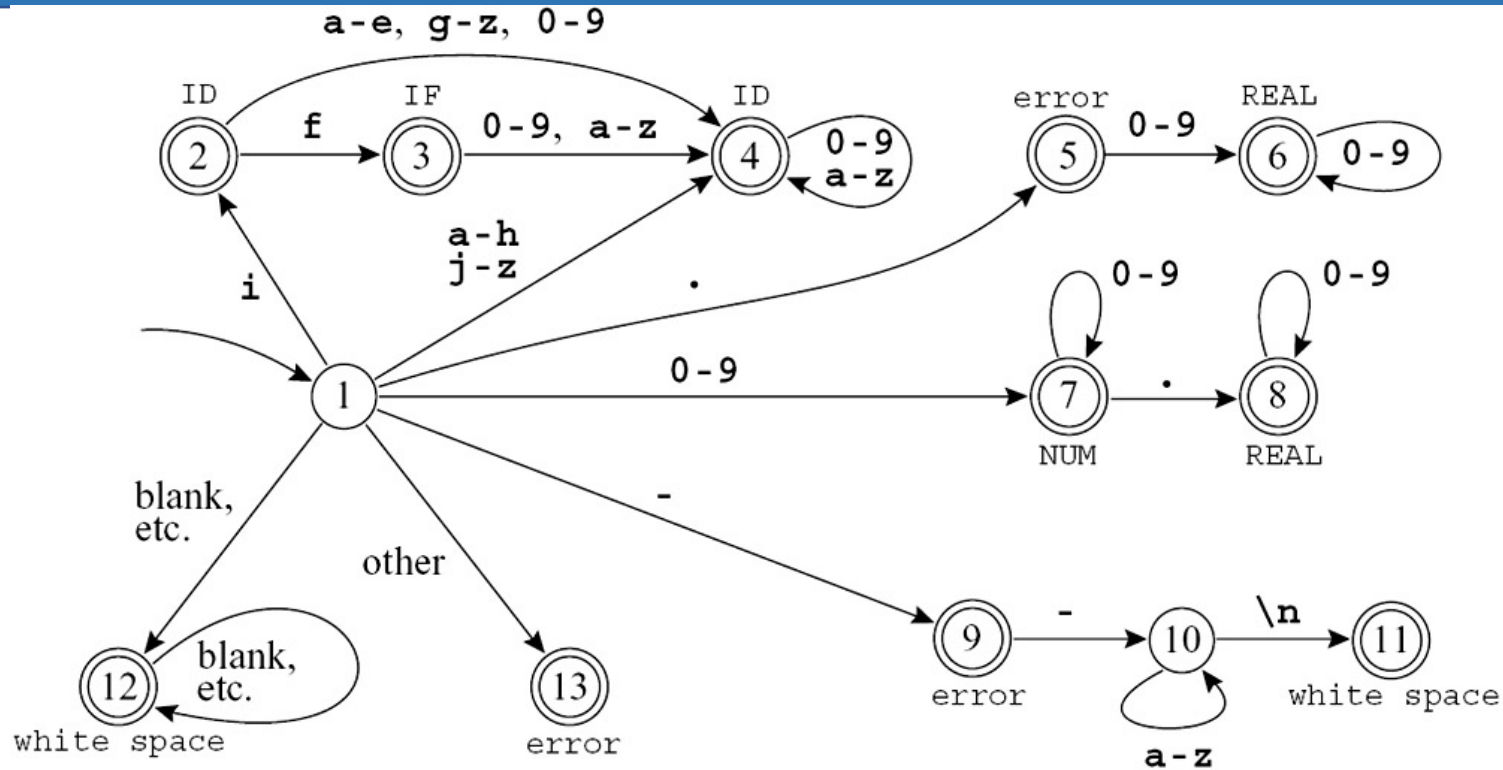


Figure 2.4

Last Final	Current State	Current Input	Accept Action
0	1	if <u>I</u> -not-a-com	
9	9	if <u>I</u> -not-a-com	
9	10	if <u>I</u> -not-a-com	
9	10	if <u>I</u> -not-a-com	
9	10	if <u>I</u> -not-a-com	
9	10	if <u>I</u> -not-a-com	
9	0	if <u>I</u> -not-a-com	error, illegal token '-'; resume

| : input position

I : automaton position

T : Input position at last final

Finite Automata

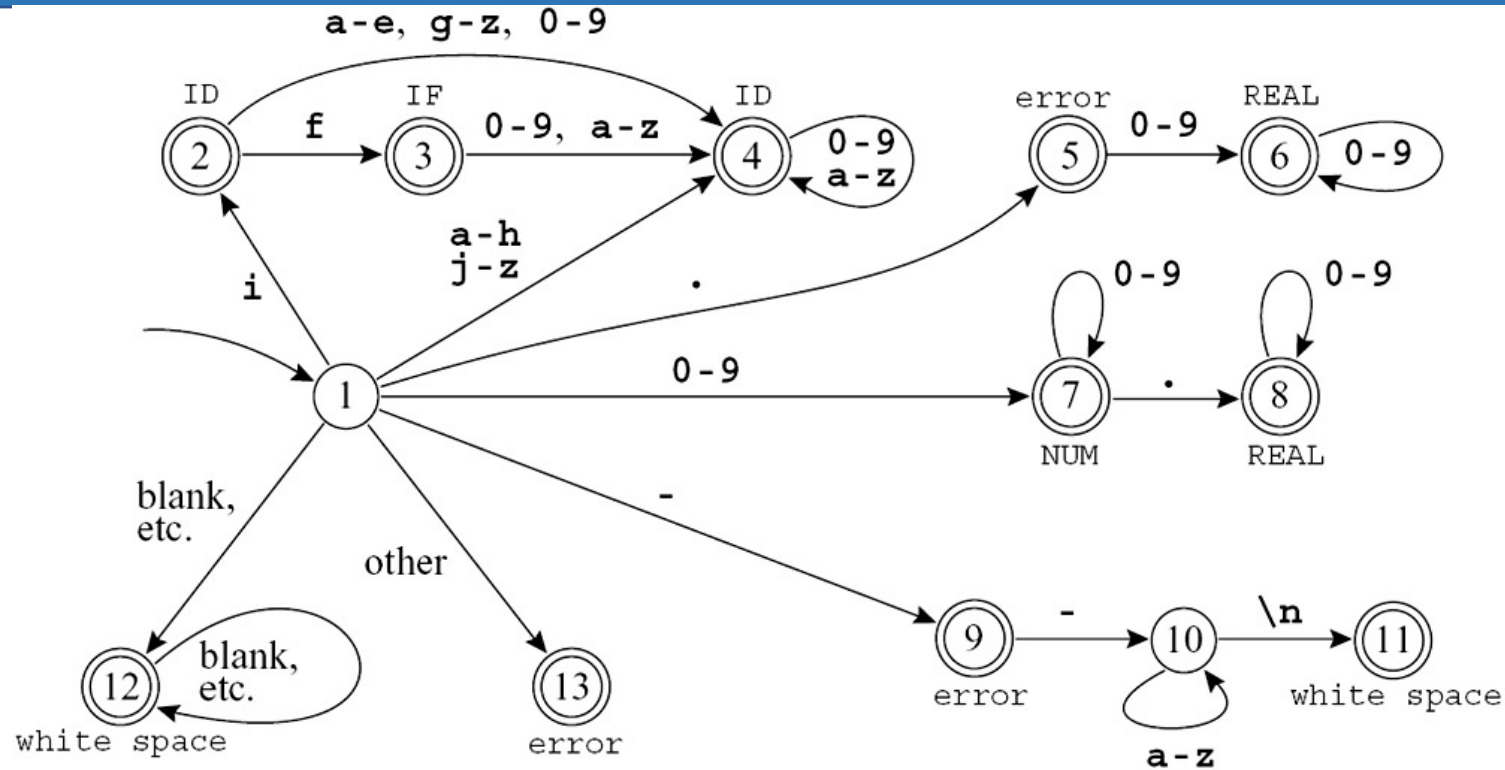


Figure 2.4

Last Final	Current State	Current Input	Accept Action	: input position ⊥ : automaton position T : last final
0	1	if ⊥ not-a-com		
9	9	if ⊥ not-a-com		
9	0	if ⊥ not-a-com	error, illegal token '-'; resume	

Finite Automata

Last Final	Current State	Current Input	Accept Action
0	1	<u>i</u> f --not-a-com	
2	2	<u>i</u> f --not-a-com	
3	3	i f <u>f</u> --not-a-com	
3	0	i f <u>f</u> _ --not-a-com	<i>return IF</i>
0	1	i f <u>f</u> --not-a-com	
12	12	i f <u>f</u> --not-a-com	
12	0	i f <u>f</u> _ --not-a-com	<i>found white space; resume</i>
0	1	i f <u>f</u> _ --not-a-com	
9	9	i f <u>f</u> _ --not-a-com	
9	10	i f <u>f</u> _ _ --not-a-com	
9	10	i f <u>f</u> _ _ _ --not-a-com	
9	10	i f <u>f</u> _ _ _ _ --not-a-com	
9	10	i f <u>f</u> _ _ _ _ _ --not-a-com	
9	0	i f <u>f</u> _ _ _ _ _ _ --not-a-com	<i>error, illegal token '-'; resume</i>
0	1	i f <u>-</u> _ --not-a-com	
9	9	i f <u>-</u> _ --not-a-com	
9	0	i f <u>-</u> _ _ --not-a-com	<i>error, illegal token '-'; resume</i>

Figure 2.5: The automaton of Figure 2.4 recognizes several tokens.

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



?

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

Outline

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression



Thompson's Construction

Non-Deterministic Finite Automata



Subset Construction, DFA Minimization

Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

2.4 Nondeterministic Finite Automata

Recall Deterministic Finite Automata

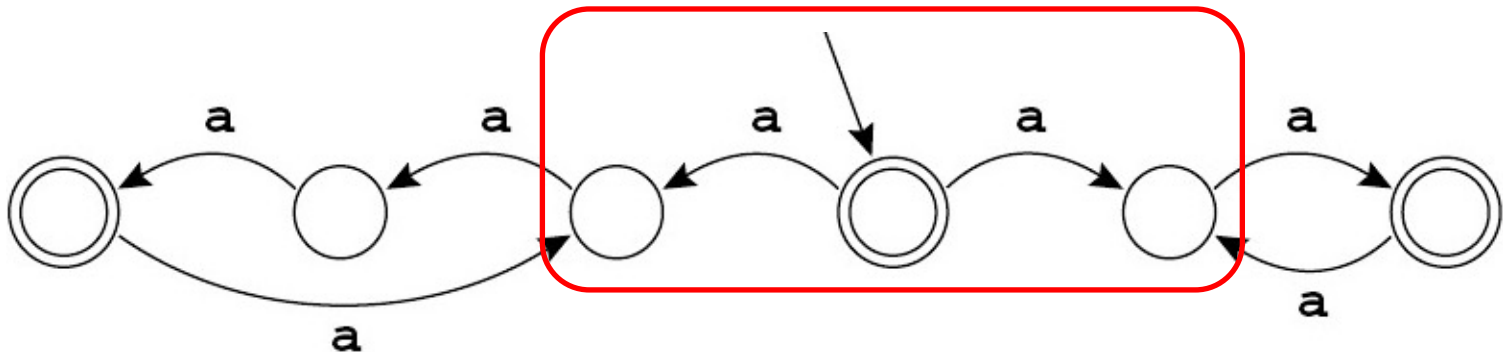
- In a *deterministic* finite automaton (DFA), **no two edges leaving from the same state** are labeled with the same symbol.

Nondeterministic Finite Automata

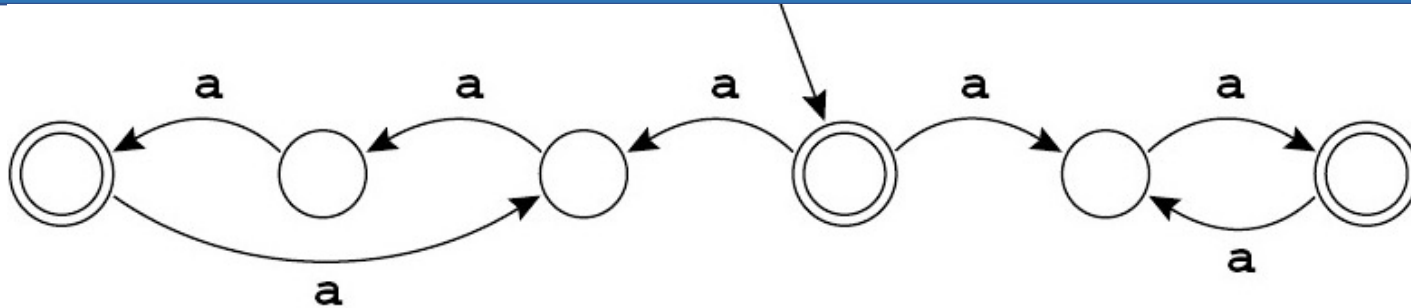
A **nondeterministic** finite automaton (NFA):

- **Have to choose** one from the edges (- labeled with the same symbol -) to follow out of a state
- Have **special edges** labeled with ϵ do not eat any symbol from input

Here is an example of an NFA:



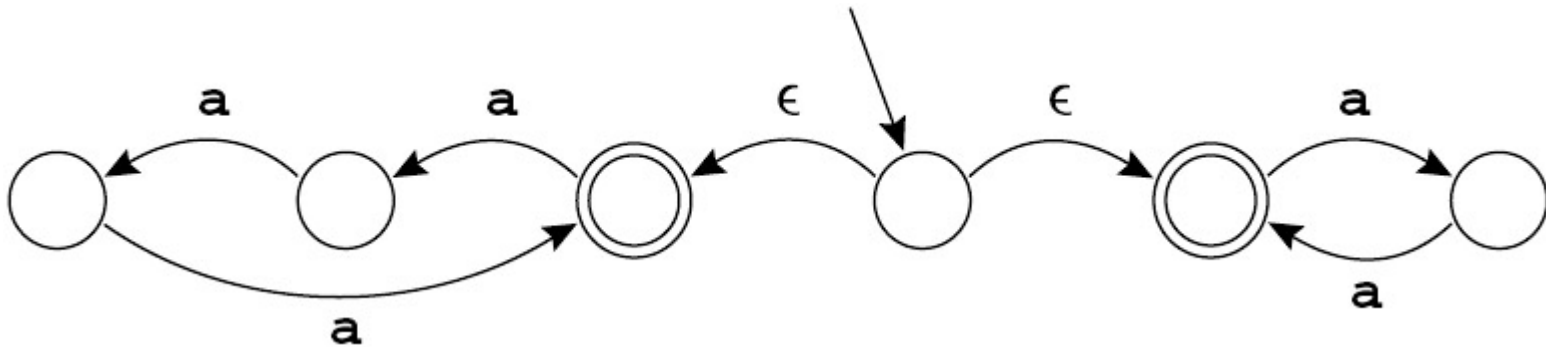
Nondeterministic Finite Automata



- How the NFA works?
- What language does the NFA recognize?
 - All strings containing a multiple of two or three a's
- On the first transition, this machine must **choose which way to go**
- It is required to accept the string if there is **any choice** of paths that will lead to acceptance.
- It **must "guess" !!** And **Always guess correctly!!**

Nondeterministic Finite Automata

Here is another NFA that accepts the same language:



Edges labeled with ϵ may be taken without using up a symbol from the input.

- The machine must choose which ϵ -edge to take !!
- A state with some ϵ -edges and edges labeled by symbols
 - Follow the corresponding symbol-labeled edge
 - Or to follow an ϵ -edge instead

Nondeterministic Finite Automata

Description of lexical tokens
(in natural language or in mind)



Manually

Regular Expression (**Static, Declarative**)



Easy

Thompson's Construction

NFA (**simulatable, quasi-executable**)



Deterministic Finite Automata



e.g., Table-Driven Implementation

Lexer

实验名称	实验内容	占比	截止日期
lab0 环境配置与测试用例编写	配置实验环境，学习SysY语法，为你的编译器编写测试用例。	5%	2025-2-23 第1周周日
lab1 词法与语法分析	实现词法分析和语法分析，将源代码转化为抽象语法树。	25%	2025-3-16 第4周周日
lab2 语义分析	实现符号表，基于抽象语法树进行语义分析。	20%	2025-4-13 第8周周日
lab3 中间代码生成	把语义分析后的抽象语法树转化为实验定义的中间代码。	25%	2025-5-11 第12周周日
lab4 目标代码生成	将中间代码转化为 RISC-V 32 汇编代码。	25%	2025-6-2 第16周周一

Exercise

- 2.1, 2.2, 2.5 (a,b), 2.6